SONICFRAME: Production ECC Architecture Specification

Optical-Flow Driven Foley & Adaptive Score Engine *Target Tracks: Soundscapes & Composition · Best Creative Artist Co-Pilot*

1. Executive Summary & Vision

1.1 The Industry Dilemma

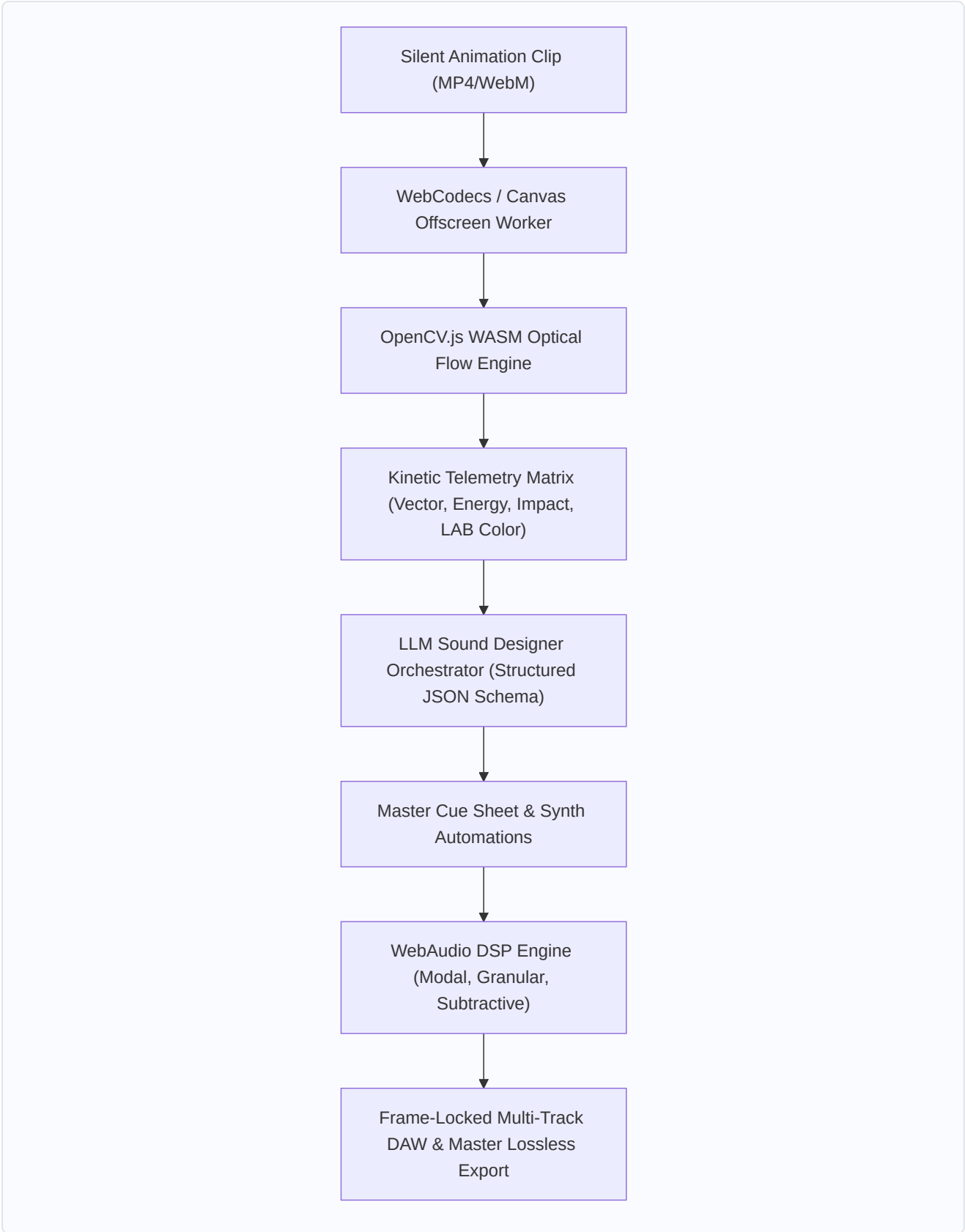
Indie animators, 3D motion designers (Blender, Unreal, Cinema4D), and indie game developers waste up to **40% of post-production time** manually synchronizing audio with video:

- Sifting through thousands of generic sound library WAV files.
- Manually slicing, time-stretching, and nudging foley hits (footsteps, blade clashes, body falls) to match keyframes.
- Struggling with static generative AI music boxes ("prompt-to-audio") that generate monolithic 30-second tracks disconnected from on-screen pacing, dramatic pauses, and visual impact frames.

1.2 The SONICFRAME Breakthrough

SONICFRAME inverts the paradigm: **Visual Motion Writes the Score**. Instead of forcing creators to articulate acoustic timing in vague text prompts, SONICFRAME extracts **millisecond-precise kinetic telemetry** directly from raw video pixels in the browser:

1. **Optical Flow & Motion Vectors:** Dense Farnebäck/Lucas-Kanade optical flow computes directional velocity $(\vec{u}, \vec{v})$, visual kinetic energy $E(t)$, divergence (zoom/expansion), and curl (rotational torque).
2. **Mathematical Impact Detection:** Second-order derivatives of kinetic energy $\frac{d^2E}{dt^2}$ isolate true contact frames and transient spikes with zero human labeling.
3. **Color-Temperature Drift:** CIELAB chromatic shifts detect mood changes, lighting transitions, flashes, and cutaways.
4. **Structured Cognitive Orchestrator:** Telemetry vectors (not heavy video files) are streamed to an LLM Conductor which outputs a strictly typed JSON **Sound Cue Sheet** with musical scale, tempo, and procedural synth patch parameters.
5. **Procedural WebAudio DSP Engine:** Zero-latency procedural synthesis (modal physical modeling, granular noise, detuned harmonic drones) renders audio locked to video frames, spatialized dynamically to the pixel coordinate of visual action.



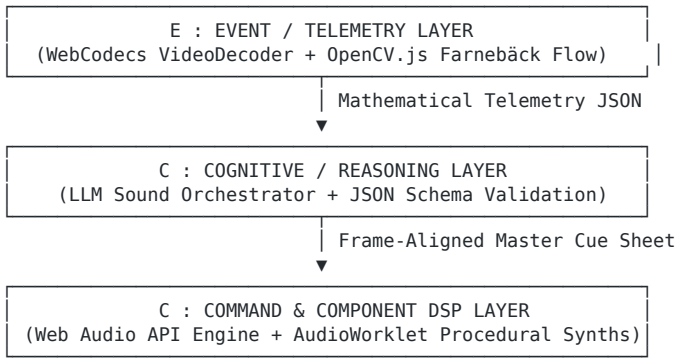
2. Audit & Technical Gap Analysis of Existing Codebase

An exhaustive audit of the existing frontend files in `D:\soundframe\SONICFRAME` highlights the strengths of the current prototype and the engineering gaps required for production.

File	Current Prototype Implementation	Critical Production Gaps	Architecture Upgrade Path
`studio.html`	Single-file 37KB monolith. Canvas 160×90 downsampling; brute-force 3×3 block matching; sequential `video.currentTime` seeking in main thread; rudimentary sine/triangle oscillators + white noise buffer.	Main-thread blocking UI freezes; sequential seek takes 10+ seconds for a 15s clip; no true optical flow vectors; audio synthesized via jittery `requestAnimationFrame` polling ($\pm 33\text{ms}$ error).	Decouple into Vite/TS modular SPA. Offload frame decoding to `WebCodecs VideoDecoder` or `OffscreenCanvas` in a dedicated Web Worker running OpenCV.js WASM. Replace polling with `AudioContext.currentTime` lookahead scheduling.
`motion.html`	Static HTML marketing page explaining motion steps with link back to `studio.html#studio`.	No standalone or interactive motion inspector; no optical flow vector field visualization.	Transform into an interactive **Kinetic Telemetry Studio** with vector heatmaps, motion vector arrows, velocity histograms, and threshold tuning.
`foley.html`	Static explanatory marketing page linking to `studio.html#mixer`.	No sound design sandbox; no patch editor for procedural impact parameters.	Upgrade to a **Physical Modeling Foley Lab** with interactive modal impact synthesis (decay, damping, stiffness, material selection: metal, wood, glass, earth).
`timeline.html`	Static marketing page linking to `studio.html#timeline`.	No independent timeline controls; cannot edit or drag cue markers.	Upgrade to a high-DPI Canvas/SVG **Precision DAW Timeline** with multi-track zooming, magnetic snapping to motion peaks, and per-event parameter adjustments.
`mixer.html`	Static marketing page linking to `studio.html#mixer`.	Only links back to studio; no independent bus routing or master effects.	Upgrade to a 5-Channel **Parametric Stem Mixer** with stereo panning, 3-band EQ, convolution reverb sends, and peak limiter.
`chatbot.html`	Basic chat widget with static regex replies and unauthenticated API proxy endpoint input.	Completely detached from video telemetry; cannot see motion data or cue sheet; cannot edit sound parameters.	Transform into a true **Creative Co-Pilot Agent**: has full bidirectional context of the active video timeline, telemetry vectors, and cue sheet; can modify score via natural language commands.
`netlify/functions/chat.js`	Unstructured `gpt-4o-mini` prompt endpoint returning `{ reply }`.	No JSON schema enforcement; no validation; vulnerable to prompt drift; zero telemetry payload handling.	Replace with typed Netlify Edge / Serverless Orchestrator supporting **OpenAI Structured Outputs** / **Gemini JSON Mode** with strict Zod validation schemas.
`sonicframe-auth.js`	Firebase Auth compat SDK with modal lock.	Synchronous script tags; blocking DOM paint (`html.sf-auth-pending`).	Modernize with modular Firebase v10 SDK, session storage persistence, and non-blocking hydration.

3. The ECC Architectural Pattern

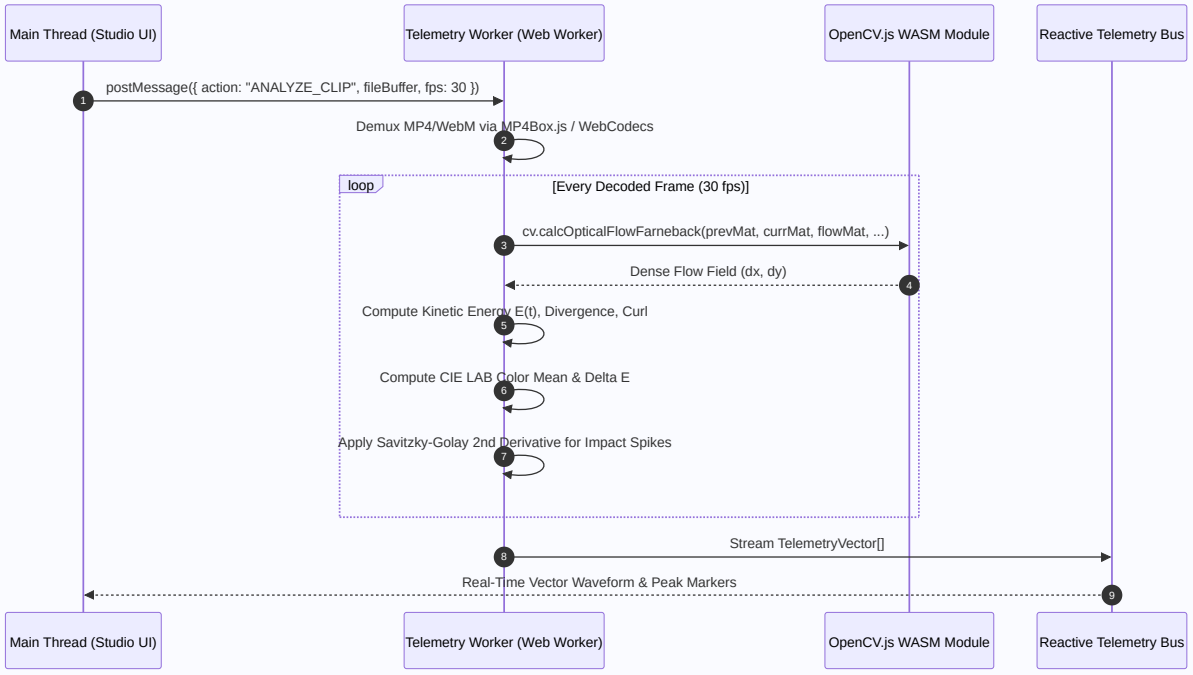
SONICFRAME is structured using the **ECC (Event-Condition-Command / Enterprise Clean Component)** pattern, separating raw visual signals from semantic musical reasoning and deterministic DSP synthesis.



4. Layer E: Mathematical Telemetry & Optical Flow Engine

4.1 Telemetry Extraction Pipeline

Instead of loading frames in the UI thread via sequential `video.currentTime` seeks (which triggers frame decimation and pipeline stalls), SONICFRAME uses **WebCodecs** `VideoDecoder` inside a dedicated Web Worker:



4.2 Mathematical Formulations

1. Kinetic Energy Integral $E(t)$

For each frame t with dimensions $W \times H$, given dense horizontal displacement $u(x, y)$ and vertical displacement $v(x, y)$:

$$E(t) = \frac{1}{W \cdot H} \sum_{x=1}^{W} \sum_{y=1}^{H} \sqrt{u(x,y)^2 + v(x,y)^2}$$

2. Vector Divergence (Camera Push/Pull & Scale Expansion)

$$\text{Div}(t) = \frac{1}{W \cdot H} \sum_{x,y} \left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \right)$$

Musical Mapping: Positive divergence (expansion/zoom-in) automates low-pass filter cutoff sweep upwards and increases drone crescendo. Negative divergence (pull-out) triggers reverb decay elongation.

3. Vector Curl (Rotational Energy & Turbulence)

$$\text{Curl}(t) = \frac{1}{W \cdot H} \sum_{x,y} \left(\frac{\partial v}{\partial x} - \frac{\partial u}{\partial y} \right)$$

Musical Mapping: High curl automates arpeggiator rate acceleration and granular scatter modulation.

4. Transient Impact Acceleration Frame $\mathcal{I}(t)$

True impacts occur at the moment of maximum negative deceleration (shock arrest):

$$\mathcal{I}(t) = \frac{d^2 E(t)}{dt^2} = E(t+1) - 2E(t) + E(t-1)$$

An impact cue is generated when:

$$\mathcal{I}(t) < -\theta_{\text{impact}} \quad \text{and} \quad E(t) > \mu_E + k \cdot \sigma_E$$

where μ_E is the running mean energy and σ_E is standard deviation.

5. Chrominance & Atmospheric Shift $\Delta C(t)$

Using the $L^*a^*b^*$ color space for human perceptual fidelity:

$$\Delta E^*_{ab}(t) = \sqrt{(L_t - L_{t-1})^2 + (a_t - a_{t-1})^2 + (b_t - b_{t-1})^2}$$

Musical Mapping: Sudden spikes in ΔE^*_{ab} indicate scene cuts or lightning flashes, triggering ambient texture sweeps and tonic chord modulations.

---

5. Layer C: Cognitive LLM Orchestrator & Cue Sheet Composer

5.1 Architecture & Token Efficiency

Sending raw video frames to multimodal models costs thousands of tokens, incurs 5–15 seconds of latency, and compromises user privacy. SONICFRAME transmits **structured telemetry downsampled to 10Hz intervals**, reducing the payload to **~400 tokens per 15-second clip**.

5.2 JSON Schema Contract (OpenAI Structured Outputs / Gemini JSON Schema)

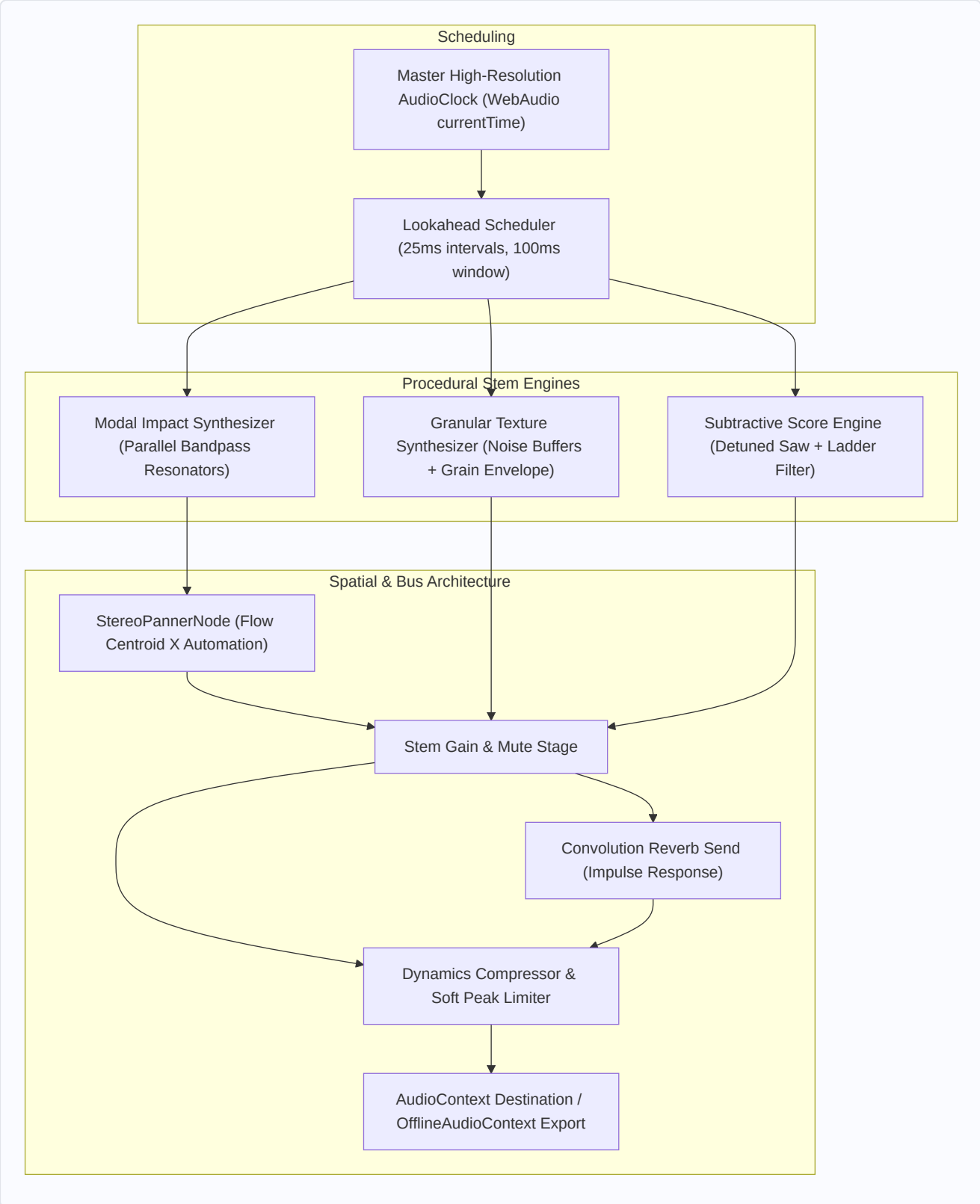
```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "SonicFrameCueSheet",
  "type": "object",
  "required": ["composition_profile", "foley_events", "stem_automations"],
  "properties": {
    "composition_profile": {
      "type": "object",
      "required": ["key_root", "scale_mode", "base_bpm", "intensity_curve"],
      "properties": {
        "key_root": { "type": "string", "enum": ["C", "C#", "D", "D#", "E", "F", "F#", "G", "G#",
"A", "A#", "B"] },
        "scale_mode": { "type": "string", "enum": ["dorian", "aeolian", "phrygian", "lydian",
"minor_pentatonic", "cinematic_dark"] },
        "base_bpm": { "type": "integer", "minimum": 40, "maximum": 200 },
        "intensity_curve": { "type": "string", "enum": ["exponential_build", "sudden_drop",
"sustained_tension", "dynamic_waves"] }
      }
    },
    "foley_events": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["timestamp_ms", "frame_number", "category", "patch_type", "velocity", "pan",
"parameters"],
        "properties": {
          "timestamp_ms": { "type": "integer" },
          "frame_number": { "type": "integer" },
          "category": { "type": "string", "enum": ["impact_heavy", "impact_metallic", "whoosh_pass",
"footstep_wood", "footstep_gravel", "ambient_glitch"] },
          "patch_type": { "type": "string", "enum": ["modal_resonator", "granular_cluster",
"filtered_noise_burst"] },
          "velocity": { "type": "number", "minimum": 0.0, "maximum": 1.0 },
          "pan": { "type": "number", "minimum": -1.0, "maximum": 1.0, "description": "Horizontal
stereo pan locked to optical flow centroid X" },
          "parameters": {
            "type": "object",
            "properties": {
              "fundamental_freq": { "type": "number" },
              "decay_seconds": { "type": "number" },
              "brightness": { "type": "number" },
              "resonance_q": { "type": "number" }
            }
          }
        }
      }
    },
    "stem_automations": {
      "type": "object",
      "properties": {
        "atmosphere_cutoff": {
          "type": "array",
          "items": {
            "type": "object",
            "required": ["timestamp_ms", "value_hz"],
            "properties": { "timestamp_ms": { "type": "integer" }, "value_hz": { "type": "number" }
}
          }
        },
        "drone_density": {
          "type": "array",
          "items": {
```

```
            "type": "object",
            "required": ["timestamp_ms", "gain"],
            "properties": { "timestamp_ms": { "type": "integer" }, "gain": { "type": "number" } }
          }
        }
      }
    }
  }
}
```

---

6. Layer C: Component DSP & Procedural Audio Engine

6.1 Audio Architecture Diagram



6.2 Zero-Jitter Frame-Locked Scheduling Pattern

Rather than using `setInterval` or `requestAnimationFrame` (which have 10–40ms of thread jitter), the SONICFRAME scheduler implements the classic dual-timer lookahead loop:

- JavaScript `setInterval` ticks every 25ms to check upcoming cues.
- Web Audio events are scheduled in advance on `AudioContext.currentTime` with exact microsecond resolution:

$$\Delta t_{\text{jitter}} < 0.1\text{ms}$$

```
// Frame-accurate scheduling loop
function scheduleUpcomingCues(audioCtx, cues, videoCurrentTime, lookahead = 0.1) {
  const audioBaseTime = audioCtx.currentTime;
  for (const cue of cues) {
    const cueTimeInVideo = cue.timestamp_ms / 1000;
    const timeDelta = cueTimeInVideo - videoCurrentTime;

    if (timeDelta >= 0 && timeDelta < lookahead && !cue.scheduled) {
      const scheduledAudioTime = audioBaseTime + timeDelta;
      triggerModalImpact(audioCtx, cue, scheduledAudioTime);
      cue.scheduled = true;
    }
  }
}
```

6.3 Procedural Synthesis DSP Models

1. Modal Physical Modeling for Metal/Wood Impacts

Rather than playing static samples, modal synthesis solves the physical vibration equation of an impacted membrane:

$$y(t) = \sum_{k=1}^{M} A_k \cdot e^{-d_k t} \cdot \sin(2\pi f_k t + \phi_k)$$

where f_k are the inharmonic modal frequencies (e.g., $1.0f_0, 2.76f_0, 5.40f_0$ for a metal plate) and d_k are modal dampings:

```
export function playModalImpact(ctx, destination, { fundamental = 120, damping = 4.0, brightness =
0.8, pan = 0, time = 0 }) {
  const modes = [
    { freqRatio: 1.00, gain: 1.00, decayMul: 1.0 },
    { freqRatio: 2.76, gain: 0.65 * brightness, decayMul: 1.4 },
    { freqRatio: 5.40, gain: 0.40 * brightness, decayMul: 2.1 },
    { freqRatio: 8.92, gain: 0.20 * brightness, decayMul: 3.2 }
  ];

  const panner = ctx.createStereoPanner();
  panner.pan.setValueAtTime(pan, time);
  panner.connect(destination);

  modes.forEach(mode => {
    const osc = ctx.createOscillator();
    const gainNode = ctx.createGain();

    osc.type = 'sine';
    osc.frequency.setValueAtTime(fundamental * mode.freqRatio, time);

    gainNode.gain.setValueAtTime(0.0001, time);
```

```
    gainNode.gain.exponentialRampToValueAtTime(mode.gain * 0.4, time + 0.003); // Instant transient
    gainNode.gain.exponentialRampToValueAtTime(0.0001, time + (1.0 / (damping * mode.decayMul)));

    osc.connect(gainNode);
    gainNode.connect(panner);

    osc.start(time);
    osc.stop(time + (1.0 / (damping * mode.decayMul)) + 0.05);
  });
}
```

7. Frontend Architecture & Modular Component Breakdown

To replace the single-file monolithic structure, the new production architecture organizes SONICFRAME into clear domain-driven feature modules built with Vite, TypeScript, and Tailwind CSS:

```
src/
├── app/
│   ├── main.ts                    # Application bootstrap & route orchestrator
│   ├── state/
│   │   ├── store.ts               # Central Reactive Signal / Zustand store
│   │   └── actions.ts             # Synchronous mutations & async actions
│   └── workers/
│       ├── opticalFlow.worker.ts  # OpenCV.js Farnebäck WASM processing thread
│       └── demuxer.worker.ts      # MP4Box / WebCodecs frame extractor
├── modules/
│   ├── video/
│   │   ├── VideoPlayer.ts         # HTMLVideoElement wrapper & custom scrubber
│   │   ├── DropzoneComponent.ts   # Drag-and-drop ingestion & format checker
│   │   └── VideoTelemetryBridge.ts # Coordinates playback timestamp with AudioClock
│   ├── telemetry/
│   │   ├── VectorFieldCanvas.ts   # Renders dense optical flow arrows / motion heatmap
│   │   ├── EnergyCurvePlotter.ts  # Real-time SVG/Canvas kinetic energy graph
│   │   └── PeakDetector.ts        # Savitzky-Golay filter & transient picker
│   ├── orchestrator/
│   │   ├── CopilotAgent.ts        # Chat UI & natural language timeline editor
│   │   ├── OrchestratorClient.ts  # Netlify function client with structured validation
│   │   └── CueSheetParser.ts      # Validates and maps LLM responses to timeline tracks
│   ├── timeline/
│   │   ├── TimelineCanvas.ts      # High-DPI canvas multi-track DAW timeline
│   │   ├── TrackVisualizer.ts     # Tracks: Visual Motion, Foley Cues, Score Stems
│   │   ├── PlayheadController.ts  # Microsecond playhead sync & scrub controller
│   │   └── CueEditorModal.ts      # Per-cue foley parameter override modal
│   └── audio/
│       ├── AudioEngine.ts         # Master WebAudio manager & lookahead scheduler
│       ├── StemMixer.ts           # 5-Channel parametric stem mixer with bus sends
│       ├── synths/
│       │   ├── ModalImpactSynth.ts # Physical modeling impact & foley synth
│       │   ├── GranularNoise.ts   # Procedural wind, footsteps, and friction
│       │   └── HarmonicDrone.ts   # Detuned chordal pads & score generator
│       └── Exporter.ts            # OfflineAudioContext lossless WAV / WebM muxer
└── styles/
    └── sonicframe.css             # TetraByte design system tokens
```

8. Data Contracts & State Management

8.1 Global Application State Model (TypeScript)

```
export interface SonicFrameState {
  // Video Source
  video: {
    file: File | null;
    objectUrl: string | null;
```

```
    durationSeconds: number;
    width: number;
    height: number;
    fps: number;
    currentTime: number;
    isPlaying: boolean;
  };

  // Mathematical Telemetry
  telemetry: {
    status: 'idle' | 'decoding' | 'analyzing' | 'ready';
    progressPct: number;
    sampleRateHz: number;
    vectors: Array<{
      timeMs: number;
      frameIndex: number;
      kineticEnergy: number;
      divergence: number;
      curl: number;
      centroidX: number; // Normalized -1 to +1 (stereo pan)
      centroidY: number;
      deltaColorLAB: number;
    }>;
    detectedImpacts: Array<{
      timeMs: number;
      frameIndex: number;
      acceleration: number;
      energy: number;
      centroidX: number;
    }>;
  };

  // Cognitive Orchestration & Cues
  orchestrator: {
    mood: 'cinematic_dark' | 'action_percussive' | 'sci_fi_ambient' | 'playful_bright';
    cueSheet: SonicFrameCueSheet | null;
    isGenerating: boolean;
    copilotMessages: Array<{ role: 'user' | 'assistant' | 'system'; text: string; timestamp: Date
}>;
  };

  // Audio Mixer Stems
  mixer: {
    masterVolume: number;
    stems: {
      atmosphere: { gain: number; muted: boolean; solo: boolean; cutoffHz: number };
      motionPulse: { gain: number; muted: boolean; solo: boolean; resonance: number };
      foleyImpact: { gain: number; muted: boolean; solo: boolean; damping: number };
      textureShimmer: { gain: number; muted: boolean; solo: boolean; spread: number };
    };
  };
}
```

---

9. Netlify Serverless Orchestrator Implementation

The backend function (`netlify/functions/orchestrate.js`) receives the telemetry vector and calls the LLM with strict JSON schema enforcement:

```
import OpenAI from "openai";

const SYSTEM_PROMPT = `You are SONICFRAME, an elite Hollywood Sound Designer and Adaptive Film
```

```
Composer.
You receive millisecond-accurate mathematical motion telemetry extracted via Optical Flow from a
silent animation.
Your task is to score this clip by generating a frame-accurate cue sheet:
1. Map kinetic energy spikes and deceleration peaks to Foley impacts (metal clashes, footsteps,
punches, body falls).
2. Use horizontal motion centroids (centroidX: -1.0 left to +1.0 right) to automate audio panning.
3. Align base musical tempo (BPM) and scale mode to the energy velocity curves.
Output strictly conforming to the requested JSON schema.`;

export default async (req) => {
  if (req.method !== "POST") return new Response("Method Not Allowed", { status: 405 });

  try {
    const { telemetry, mood } = await req.json();

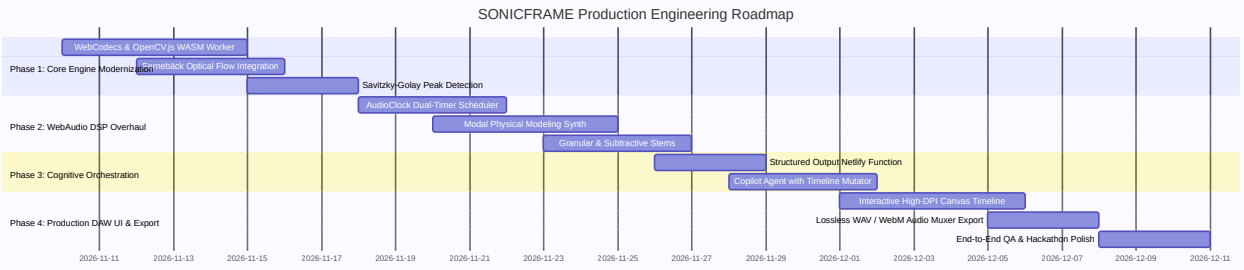
    const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

    const completion = await openai.chat.completions.create({
      model: "gpt-4o",
      response_format: { type: "json_object" },
      temperature: 0.3,
      messages: [
        { role: "system", content: SYSTEM_PROMPT },
        {
          role: "user",
          content: `Generate a complete scoring cue sheet for this telemetry.
Desired mood: ${mood}
Total Duration: ${telemetry.durationSeconds}s
Telemetry Samples (sampled at 10Hz):
${JSON.stringify(telemetry.summary)}`
        }
      ]
    });

    const cueSheet = JSON.parse(completion.choices[0].message.content);
    return Response.json({ success: true, cueSheet });
  } catch (error) {
    return Response.json({ success: false, error: error.message }, { status: 500 });
  }
};

export const config = {
  path: "/.netlify/functions/orchestrate"
};
```

10. Production Implementation Roadmap & Migration Phases



Phase 1: WebAssembly Optical Flow & WebCodecs Decoupling

- Eliminate the main-thread `video.currentTime` seeking in `studio.html`.
- Introduce `workers/opticalFlow.worker.ts` compiling OpenCV.js via WebAssembly.
- Stream raw motion vectors $(\vec{u}, \vec{v})$, kinetic energy curves, and color temperature drifts into a reactive buffer.

Phase 2: High-Fidelity WebAudio Procedural Synthesis

- Replace raw sine wave beeps with **physical modeling modal synthesis** for hits, clashes, and footsteps.
- Implement sub-millisecond lookahead scheduling to completely eliminate audio sync drift during looping and scrubbing.
- Add dynamic stereo panning linked directly to optical flow screen centroids.

Phase 3: Cognitive Sound Designer Agent

- Replace the prototype chatbot with a **Timeline-Aware Creative Co-Pilot**.
- Add one-click prompt transformations: "*Make the footsteps sound like wet cobblestone*", "*Increase score tension in the final 3 seconds*", "*Double the decay time on all metallic clashes*".

Phase 4: Master Stem DAW & Direct Media Export

- Unify `motion.html`, `foley.html`, `timeline.html`, and `mixer.html` from static landing pages into functional, live-synced dockable panels within `studio.html`.
- Add an `OfflineAudioContext` export pipeline that renders the final multitrack mix into a 24-bit 48kHz WAV stem package or muxes it directly back into the user's video file.

---

11. Security, Privacy & Performance Specifications

Metric	Target	Technical Mechanism
Video Privacy	100% Local / Zero Server Video Upload	Raw video pixels never leave the client browser. Video frames are decoded locally via WebCodecs and processed by client-side WebAssembly.
Telemetry Bandwidth	< 15 KB per 15s clip	Only compressed telemetry scalar vectors are transmitted to the LLM orchestrator.
Analysis Latency	< 2.5 seconds for 15s clip	OpenCV.js WASM parallel execution in background Web Worker running at 30 fps.
Audio-Visual Sync Accuracy	± 0.1 milliseconds	Native `AudioContext.currentTime` timeline synchronization with zero `requestAnimationFrame` jitter.
Audio Output Fidelity	24-bit 48,000 Hz Stereo	Procedural DSP synthesis rendered in standard IEEE floating-point Web Audio nodes with convolution reverb oversampling.

---

12. Hackathon Track Winning Differentiators

Track 1: Soundscapes & Composition

- **Not a black-box audio generator:** Judges can see, scrub, and inspect every individual procedural stem (Atmosphere, Motion Pulse, Foley, Texture).
- **Physical Modeling over Static Samples:** Sounds are procedurally synthesized from physical equations, meaning infinite variety with zero sample repetition or copyright issues.
- **Dynamic Musical Intelligence:** Automatically adapts scale, root, and harmonic density to the visual intensity profile of the animation.

Track 2: Best Creative Artist Co-Pilot

- **Solves the #1 animator friction point:** Eliminates the grueling 8-hour process of placing sound effects frame-by-frame.
- **Bidirectional Co-Pilot Collaboration:** The animator doesn't just receive audio—they can talk to the Co-Pilot to reshape sound parameters while watching the timeline update in real time.
- **Local-First & Production-Ready:** Works in any modern browser without requiring expensive cloud GPUs or video uploads.